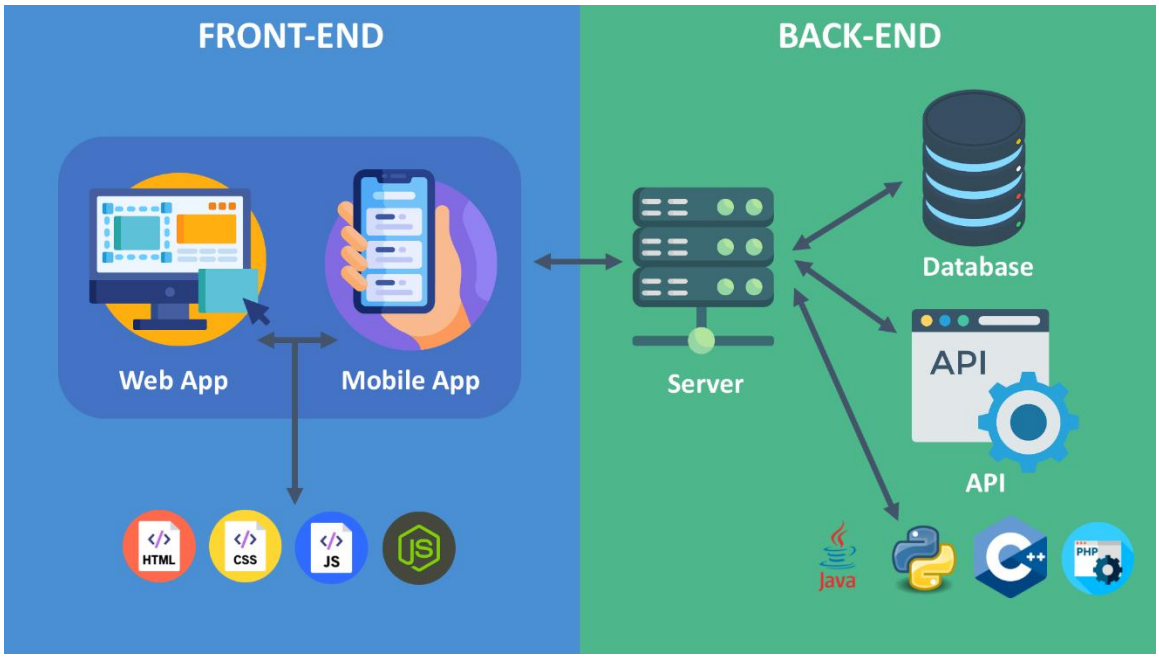
INSTITUTO UNIVERSITÁRIO DE LISBOA



Doc. João Miguel Matos e Doc. Catarina Ferreira Silva

Trabalho realizado por: Manuel Almeida n111176, Dinis Prata n110897, Edgar Quintino n99150

- Resumo

1. Introdução

O objetivo deste projeto é desenvolver uma REST API totalmente funcional de suporte e uma aplicação Front-end. A API permitirá aos utilizadores realizar operações CRUD (Create, Read, Update, Delete).

2.1. Modelação da base de dados

Foram feitas diversas alterações aos datasets:
1. Foram removidas as reviews de filmes do dataset Users.

2. Foram adicionadas reviews aos eventos no dataset Users.

3. Foi acrescentado um parâmetro de "imageURL", para se inserir um link com a fotografia do User.

4. Foi alterado o parâmetro "id" para que todos os values apresentados sejam inteiros (visto que alguns estavam em string).

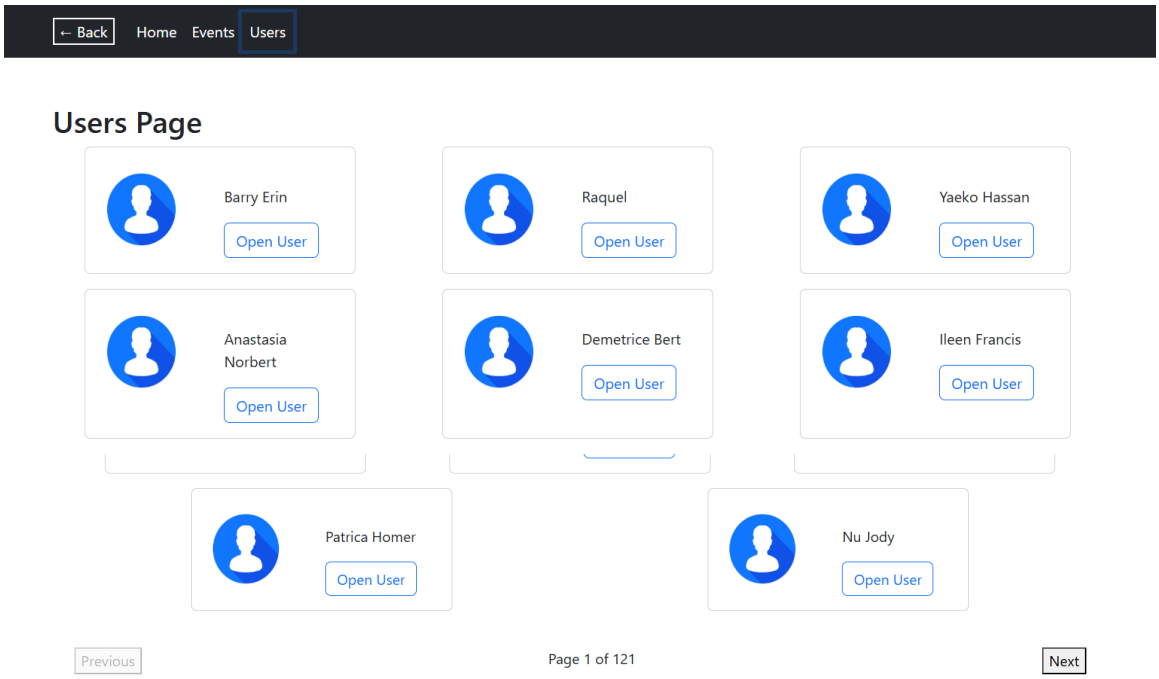
3. Implementação

3.1. Funcionalidades principais

Funcionalidades principais de Users:

Primeiramente criamos a pagina Users onde listamos todos os users da nossa User collections.

Esta página apresenta no seu topo os botoes de navegação para o Home, a pafina de Events e a prórpia página de Users, assim como o Universal button "Back" para retornar para a página anterior. De seguida podemos observar a lista de users completa assim como botões para navegar em diferentes páginas de users pois cada página do site suporta 50 users de cada vês.

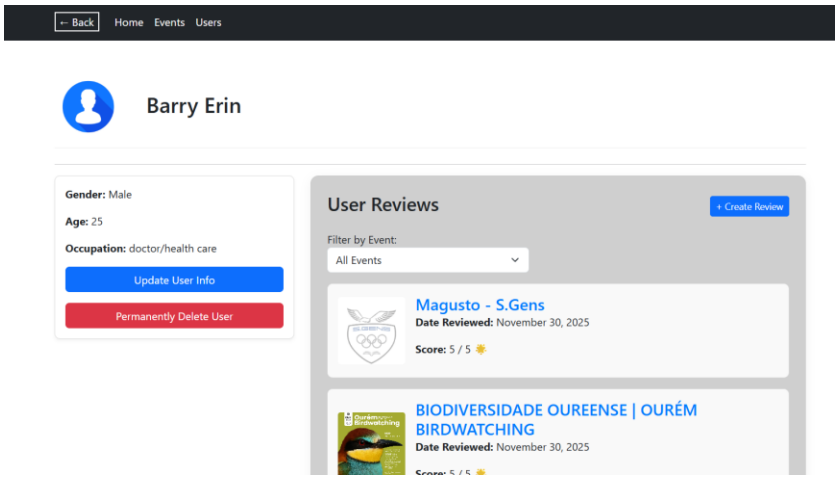


Cada elemento listado é um user card onde colocamos uma imagem genérica, caso o user não tenha uma foto de perfil na base de dados, ou a sua própria foto caso ela se encontre guardada assim como o seu nome.

Funcionalidades principais de User Profile:

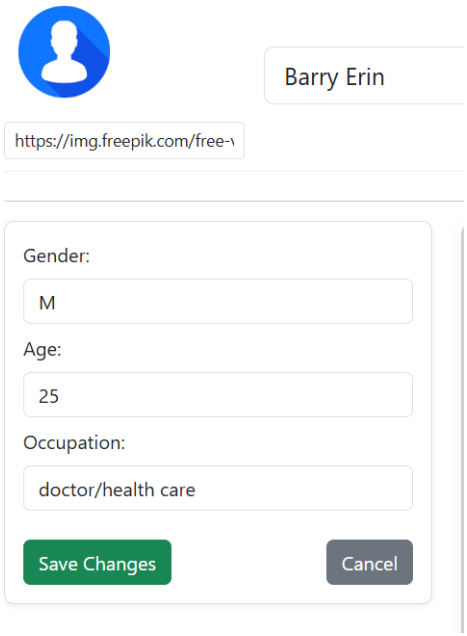
Aqui temos as opções de navegar entre as páginas de Users, como podemos também abrir um namecard ou criar um User.
Para a implementação deste último ponto foi necessário criar um componente, CreateUserModal.js, em que definimos o que acontece ao clicar no botão. Foi necessário também importar para o Users.js esse componente, como criar uma função para gerir esta funcionalidade e criar lá o componente modal.

Ao selecionarmos um dos namecards da lista de Users entramos na página de perfil do User selecionado, nesta página podemos encontrar as informações dos users assim como alguns botões para editar/apagar/criar informação e uma lista com as diversas reviews de eventos criados pelo user.

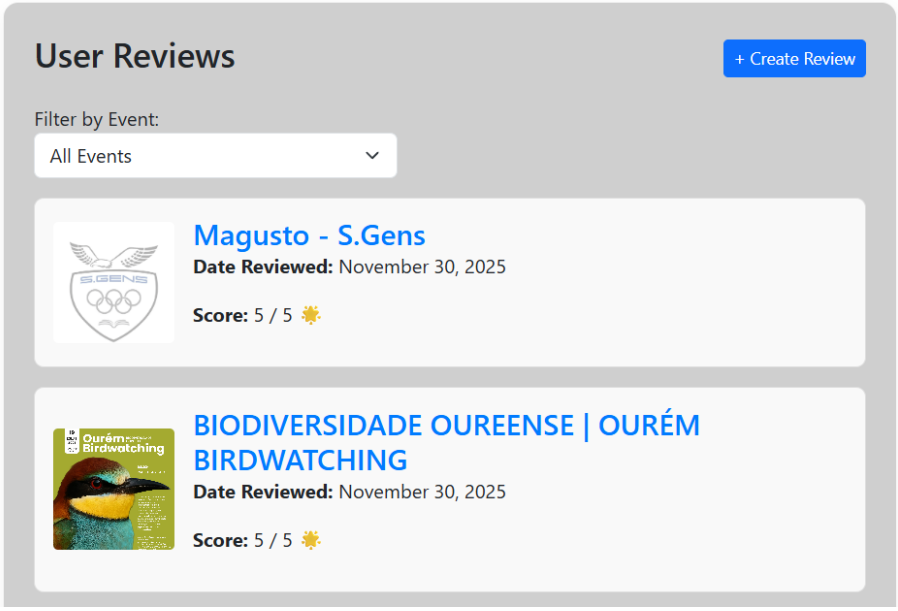


Do lado esquerdo temos as informações pessoais do user, o botão de Update e o botão de Delete.

As informações podem ser editadas com o botão de update que ao ser clickado permite que o utilizador edite todos os seus campos assim como a foto de perfil, faz aparecer o botão de Save Changes para guardar as mudanças e o botão Cancel para cancelar a ação.



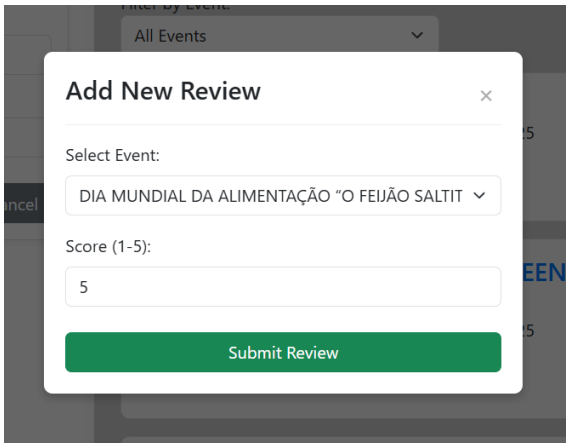
Do lado direito temos a lista de reviews criada pelo utilizador assim como o botão para criar novas reviews e um filtro para filtrar a lista.



Esta lista foi feita com a componente de react ReviewList.js que cria uma lista de review-item-cards compostos pela imagem associada ao evento, o título onde podemos clickar para entrar na página do evento, a data a que a review foi criada e o score da review.

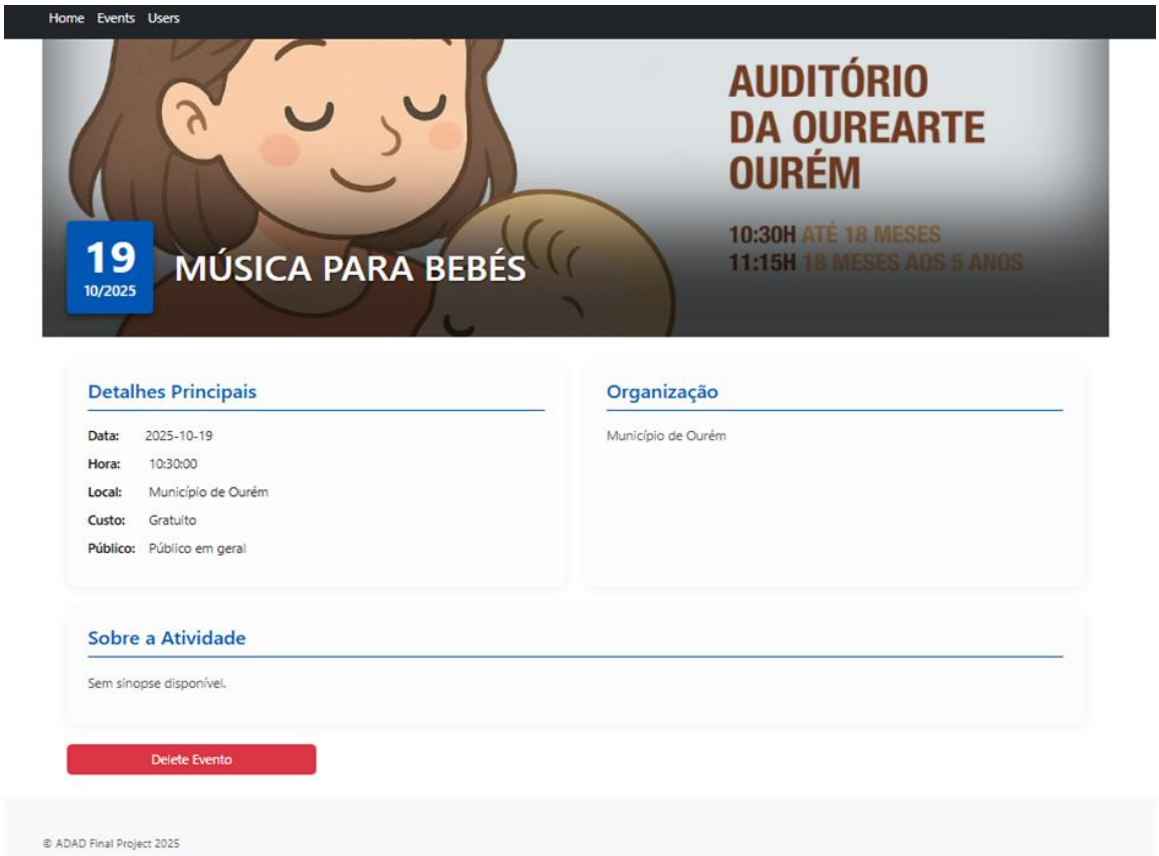
Podemos filtrar a lista para apenas apresentar eventos específicos com o dropdown Filter by Event que ao selecionar um evento apenas as reviews desse evento vão aparecer.

Por fim é possivel criar uma nova review através do botão Create Review que abre uma nova componente de react CreateReviewModal, onde podemos selecionar um evento, de uma lista dropdown de todos os eventos do backend, e escolher o score que queremos atribuir ao evento. Ao clickar no botão submit review a review é guardada no backend e aparace na lista de reviews do User.



Funcionalidades Principais – Event Profile

Ao selecionar um evento da página dos eventos, o utilizador é encaminhado para a página do evento escolhido. Esta página apresenta a informação detalhada do evento, permitindo consultar todos os dados relevantes de forma organizada e visualmente apelativa.



A secção superior da página contém três secções principais: Detalhes Principais, Organização e Sobre a Atividade.

Estas três secções onde respetivamente apresenta os dados essenciais do evento, quem é a entidade responsável pela organização e uma pequena sinopse sobre o evento

Adicionalmente no final da página integra uma funcionalidade de botão de “Delete Evento”, que permite remover o evento diretamente.
Esta página segue uma estrutura modular definidos em EventDetails.css.

Funcionalidade Principais – Events

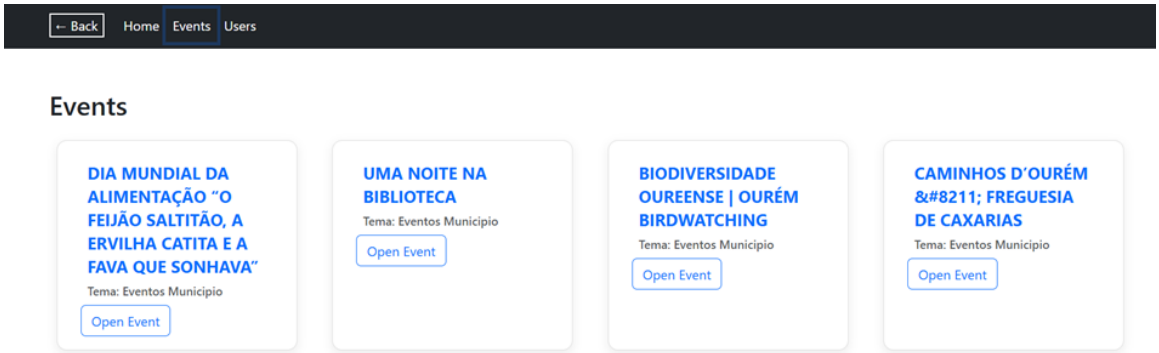
A página Events apresenta todos os eventos disponíveis no sistema organizados em páginas. Cada página uma apresenta uma grelha de eventcards. C individual e inclui:

- O título do evento, apresentado em destaque;

- O tema associado ao evento;

-Um botão “Open Event”, que permite aceder à página detalhada do evento selecionado.

Esta página segue uma estrutura visualmente uniforme com implementação de paginação definidos em EventCard.css



3.2. Estrutura do código

- Organização de pastas

O nosso projeto é composto por duas pastas focais, COMPONONENTS e PAGES.

Components

Nesta pasta é onde se encontram todos os componentes de react desenvolvidos no frontend do projeto

-EventCard e UserCard:

- Estas componentes apresentam dinamicamente as informações dos Eventos e dos Users em pequenos cartões. Ambas aceitam um argumento “props” que armazena as informações do evento e do user que vão ser colocados nos cards.
- O UserCard difere mais pois apresenta uma imagem base genérica que é colocada em todos os users caso eles não apresentem uma imagem de perfil

-Footer, Navigation e UniversalBackButton:

- O Footer do nosso projeto é muito simples e apenas apresenta a sigla da cadeira e do projeto assim como uma data dinâmica do ano atual

```
return (
    <footer className="footer bg-light pt-5 pb-5">
        <div className="container">
            <div className="row">
                <p className="text-muted small mb-4 mb-lg-0">&copy; ADAD Final Project {new Date().getFullYear()}</p>
            </div>
        </div>
    </footer>
);
```

- Na Navigation component é onde se encontram os botões de navegação principais do projeto que permitem aos utilizadores facilmente visitarem diferentes páginas do projeto:

```
return (
        <Navbar collapseOnSelect expand="sm" bg="dark" variant="dark">
            <div className="container">
                <UniversalBackButton />
                <Navbar.Toggle aria-controls="navbarScroll" data-bs-toggle="collapse" data-bs-target="#navbarScroll"
                <Navbar.Collapse id="navbarScroll">
                    <Nav>
                        <NavLink  eventKey="1" as={Link} to="/">Home</NavLink>
                        <NavLink  eventKey="2" as={Link} to="/events">Events</NavLink>
                        <NavLink  eventKey="3" as={Link} to="/users">Users</NavLink>
                    </Nav>
                </Navbar.Collapse>

            </div>
        </Navbar>
);
```

  - Home button para voltar ao início do site
  - Events button para ir para a página /events com a lista de eventos
  - Users button para ir para a página /users com a lista de users
  - Back button (UniversalBackButton) para voltar para a página anterior do site

- A componente do UniversalBackButton é responsável pelo botão de voltar para a página anterior sendo que quando estamos na página Home não faz sentido termos o botão back então tornamo-lo null para o omitir.

```
const isHomePage = location.pathname === '/';


const goBack = () => {
    navigate(-1);

};


if (isHomePage) {
    return null;
}

return (
    <button
        onClick={goBack}
        className="universal-back-button"
        aria-label="Go back to the previous page"
    >
        ← Back
    </button>
);
```

-ReviewList e CreateReviewModal:

- Estas componentes estão relacionadas com a listagem e criação de reviews dinamicamente, **ReviewList** é a componente que apresenta a lista de reviews e **CreateReviewModal** é a componente onde o user pode criar uma nova review para inserir na base de dados.

- Começando pela **ReviewList** começamos por criar diferentes variavéis de estado para armazenar informação:

    - **ReviewsWithEvent:** Variável que armazena as reviews retiradas do backend;

    - **FilteredReviews:** A lista de reviews que aparece no frontend consoante o filtro aplicado, caso seja All Events, a lista contêm todos os reviews do user;

    - **Events:** Aqui é onde vamos armazenar a lista de eventos unicos utilizada no dropdown de filtros;

    - **SelectedEvent:** Onde é armazenado o filtro/evento que foi selecionado;

    - **IsModalOpen:** Controla a visibilidade da janela CreateReviewModal para quando for necessário criar uma review;

- De seguinda utilizamos um fetchEvents dentro do useEffect onde através do GET endpoint `` `http://localhost:3000/events/${review.eventId}` `` extraimos todas as informaçoes do evento que está associado a cada review através do eventId

associado à review.

```
const fetchEvents = async () => {
    const enrichedReviews = await Promise.all(
        eventReviews.map(async (review) => {
            try {
                const response = await fetch(`http://localhost:3000/events/${review.eventId}`, {
                    method: 'GET',
                    headers: { 'Content-Type': 'application/json' },
                });
                const eventData = await response.json();
                return {
                    ...review,
                    nome_atividade: eventData.nome_atividade || `Event ${review.eventId}`,
                    imageUrl: eventData.fotografia || null,
                };
            } catch (err) {
                console.error("Error fetching event:", err);
                return { ...review, nome_atividade: `Event ${review.eventId}`, imageUrl: null };
            }
        })
    );
```

- Um handleFilterChange que filtra a lista baseada no nome_atividade de cada review, ou seja, se nenhum filtro for escolhido apresenta todas as reviews, se escolher um dos nomes dos eventos vai filtrar a lista de reviews para apenas aparecerem reviews desse evento;

- Um handleReviewSuccess que é usado após uma review ser criada com sucesso, para tornar invisível o CreateReviewModal. De seguida, o componente ReviewList recebe na página User.js a função onReviewCreated, que alerta o parente do componente, ou seja, a página User para fazer um refresh da lista de reviews para que as novas reviews apareçam assim que são criadas.

```
const handleReviewSuccess = () => {
    setIsModalOpen(false);
    if (onReviewCreated) {
        onReviewCreated();
    }
};
```

- Utilizamos um .map() loop para criar um ReviewItem para cada review que existe na variável filteredReviews. Passamos cada informação da review (nome do evento, score, imagem ID) para o ReviewItem e associamos-lhe uma key para que o react consiga atualizar dinamicamente a lista de reviews composta por ReviewItems;

```
{reviewsWithEvent.length === 0 ? (
    <p>No reviews yet.</p>
) : (
    <div className="review-cards-container" >
        {filteredReviews.map((review, index) => (
            <ReviewItem
                key={`${review.eventId}-${index}`}
                eventName={review.nome_atividade}
                reviewDate={review.date}
                score={review.score}
                imageURL={review.imageUrl}
                eventId={review.eventId}
            />
        ))}
    </div>
)}
```

```
// Component to display a single review item
const ReviewItem = ({ eventName, reviewDate, score, imageURL, eventId }) => {
    // Helper function to format the date
    const formatDate = (dateString) => {
        const options = { year: 'numeric', month: 'long', day: 'numeric' };
        return new Date(dateString).toLocaleDateString(undefined, options);
    };
```

- Por fim, temos o botão que abre o CreateReviewModal ao ser clickado, assim como a secção que renderiza a componente em si :

```
{/* Button to open the modal */}
{userId && (
    <button
        className="btn btn-primary btn-sm"
        onClick={() => setIsModalOpen(true)}
        + Create Review
    </button>
)}
```

```
{/* The Modal Component */}
<CreateReviewModal
    isOpen={isModalOpen}
    onClose={() => setIsModalOpen(false)}
    onReviewSuccess={handleReviewSuccess}
    userId={userId}
/>
```

- Passando agora para a componente **CreateReviewModal,** primeiramente fazemos um fetch com um endpoint GET novamente aos eventos para extrair os eventos únicos para uma lista de dropdown que vai ser usada para escolher qual o evento queremos dar a review.

- Temos uma opção para escolher o score que queremos dar na review:

```
{/* Score Input */}
<div className="mb-4">
    <label htmlFor="score-select" className="form-label">Score (1-5):</label>
    <input
        id="score-select"
        type="number"
        className="form-control"
        value={score}
        min="1"
        max="5"
        onChange={(e) => setScore(e.target.value)}
        required
    />
</div>
```

- E por fim utilizamos um handleSubmit que através do endpoint POST `http://localhost:3000/users/${userId}/review/${selectedEventId}` publicamos a review na base de dados do user e fechamos o modal com a função onReviewSuccess().

```
const endpoint = `http://localhost:3000/users/${userId}/review/${selectedEventId}`;

try {
    const response = await fetch(endpoint, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ score: parseInt(score) }),
    });

    const data = await response.json();

    if (!response.ok) {
        throw new Error(data.message || 'Failed to submit review.');
    }

    alert(data.message || 'Review added successfully!');
    onReviewSuccess(); // Close modal and trigger data refresh in parent
} catch (err) {
    console.error("Review Submission Error:", err);
    setError(err.message || 'There was an error saving your review.');
} finally {
    setIsLoading(false);
}
```

-CreateUserModal:

- Este componente tem as funções de: Popup de registo, em que assegura que ao clicar no botão Create User aparece a janela que vai receber as informações do user.

- Seguidamente tem também a função de enviar os dados recebidos no POST para o servidor.

- E função de retornar feedback, se a operação correu com sucesso ou não.

```
{/* --- CAMPO DO ID MANUAL --- */}
<div className="mb-3 p-2 bg-light border rounded">
    <label className="form-label fw-bold">ID (Obrigatório):</label>
    <input
        type="number"
        name="customId"
        className="form-control"
        value={formData.customId}
        onChange={handleChange}
        placeholder="ex: 7001"
        required
    />
    <small className="text-muted" style={{fontSize: '0.8rem'}}>
        Escolha um número que ainda não exista na base de dados.
    </small>
</div>
```

Aqui está representado como foi feito para o campo do id do User. Usando-se o mesmo raciocínio para os seguintes parâmetros.

Pages

User.js

Na página do User temos algumas funcionalidades que estão representadas nas seguintes classes:

```
const getUser = async (userId) => {
    try {
        const response = await fetch(`http://localhost:3000/users/${userId}`);
        const data = await response.json();
        console.log(data);
        setUser(data);
    } catch (error) {
        console.error('Error fetching user:', error);
    }
};
```

É uma função assíncrona que faz um pedido GET à API para obter os dados de um utilizador específico. Se bem-sucedida, guarda esses dados no estado, user.

```
const deleteUser = async () => {
    if (!window.confirm(`Are you sure you want to delete user with ID: ${id}?`)) {
        return;
    }

    try {
        const response = await fetch(`http://localhost:3000/users/${id}`, {
            method: 'DELETE',
        });

        if (!response.ok) {
            throw new Error('Failed to delete user.');
        }

        alert(`User with ID ${id} deleted successfully!`);
        navigate('/users');
    } catch (error) {
        console.error('Deletion Error:', error);
        alert(`Error deleting user: ${error.message}`);
    }
};
```

É quem gere a remoção de um utilizador. Primeiro pede confirmação, se for confirmado, envia um pedido de DELETE à API. Se tiver sucesso, redireciona o navegador para a lista geral de utilizadores (`/users`) .

```
const handleUpdateSubmit = async (e) => {
    e.preventDefault();

    try {
        const response = await fetch(`http://localhost:3000/users/${id}`, {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(formData),
        });

        if (!response.ok) {
            throw new Error('Failed to update user.');
        }
        await getUser(id);
        setIsEditing(false);

        alert('User profile updated successfully!');

    } catch (error) {
        console.error('Update Error:', error);
        alert(`Error updating user: ${error.message}`);
    }
};
```

Esta função é acionada ao submeter o formulário de edição. Envia um pedido PUT à API com os dados alterados (formData). Se tiver sucesso, recarrega os dados do utilizador (getUser) e fecha o modo de edição.

```
const formatGender = (genderCode) => {
    if (!genderCode) return "Not available";
    const code = genderCode.toUpperCase();

    if (code === "M") return "Male";
    if (code === "F") return "Female";
    return "Other / Unspecified";
};

const handleFormChange = (e) => {
    setFormData({
        ...formData,
        [e.target.name]: e.target.value,
    });
};

const handleEditClick = () => {
    // Synchronize formData with the current user data
    setFormData({
        name: user.name || '',
        gender: user.gender || '',
        age: user.age || '',
        occupation: user.occupation || '',
        imageUrl: user.imageUrl || DEFAULT_IMAGE_URL,
    });
    setIsEditing(true);
};

const cancelEdit = () => {
    setIsEditing(false);
};
```

Aqui estão representadas as funções auxiliares usadas:

- formatGender: recebe um código ("M", "F") e retorna o texto completo legível para apresentar no ecrã (“Male”,”Female”)
- handleFormChange: Permite que o formulário seja editável. Atualiza o estado, formData, em tempo real sempre que o utilizador escreve num campo de texto (input).
- handleEditClick: Prepara o modo de edição. Copia os dados atuais do utilizador para o estado do formulário (formData) e altera o estado, isEditing, para true, fazendo aparecer os campos de input.
- cancelEdit: Altera o estado isEditing para false, fechando o formulário sem enviar quaisquer dados para a API.

Users.js

Na página Users decidimos criar as seguintes variáveis:

- Users: array com os dados dos users recebidos pelo backend;
- CurrentPage: guarda a página a que o utilizador está;
- TotalPages: indica quantas páginas existem no total através do backend;
- Limit: Limite de users que aparece por página;

Usamos a função getUsers para realizar um fetch e extrair do backend todos os users com o GET `http://localhost:3000/users?page=${page}&limit=${limit}` e colocamos os users na variável user com o setUsers().

Por fim temos o return onde mapeamos a variavel users, colocamos os users na componente de react UserCard para que o react organize as suas informações:

```
<CardGroup>
        <Row xs={1} md={2} className="d-flex justify-content-around">
        {users && users.map((user) => {
            return (
                <UserCard
                    key={user._id}
                    {...user}
                />
            );
        })}
        </Row>
    </CardGroup>
```

e no final da página os controlos de paginação com os botões para mudarmos de página na lista de users do backend.

```
<button
  disabled={currentPage <= 1}
  onClick={() => setCurrentPage(prev => prev - 1)}
>
  Previous
</button>

<span>Page {currentPage} of {totalPages}</span>

<button
  disabled={currentPage >= totalPages}
  onClick={() => setCurrentPage(prev => prev + 1)}
>
  Next
```

Event.js

A página Event.js é responsável por apresentar a informação detalhada de um evento específico, obtido dinamicamente através do seu ID.
No useEffect onde é feita o carregamento da página é feita um GET ao endpoint/events/:id onde vai buscar o evento, através do ID, e os detalhes mais importantes onde depois é feita a interface que é organizada em três secções: Detalhes Principais, Organização e Sobre a Atividade.

A página integra ainda uma funcionalidade de eliminação do evento, implementada através de um botão. Esta opção desencadeia um pedido DELETE, acompanhado de uma confirmação prévia. Após uma eliminação bem-sucedida, o utilizador é redirecionado para a página geral de eventos usando navigate(‘/events´)

```
const deleteEvent = async () => {
    if (!window.confirm(`Are you sure you want to delete event with ID: ${id}?`)) {
        return;
    }

    try {
        const response = await fetch(`http://localhost:3000/events/${id}`, {
            method: 'DELETE',
        });

        if (!response.ok) {
            throw new Error('Failed to delete event.');
        }

        alert(`Event with ID ${id} deleted successfully!`);
        navigate('/events');
    } catch (error) {
        console.error('Deletion Error:', error);
        alert(`Error deleting event: ${error.message}`);
    }
};
```

Events.js

A Página events é responsável por apresentar todos os eventos disponíveis no sistema. O use Effect vai carregar a página é chama a função getEvents onde vai buscar os eventos com a implementação de paginação com um limite de 8 cards por página. Cada card representa um evento individual e inclui o nome do evento e o tema.

```
const [events, setEvents] = useState([]);
const [currentPage, setCurrentPage] = useState(1);
const [totalPages, setTotalPages] = useState(1);
const limit = 8;

const getEvents = async (page=1) => {
  try {
    const response = await fetch(`http://localhost:3000/events?page=${page}&limit=${limit}`, {
      method: 'GET',
      headers: {
      'Content-Type': 'application/json'
      },
    });


    const data = await response.json();
    console.log(data)
    setEvents(data.eventos || []);
    setCurrentPage(data.currentPage || 1);
    setTotalPages(data.totalPages || 1);

  } catch (error) {
    console.error('Error fetching events:', error);
  }
}
```

6. Conclusões e Trabalho Futuro

- Conclusões gerais

O desenvolvimento deste projeto permitiu concretizar com sucesso a criação de uma REST API funcional, servindo como a espinha dorsal para a aplicação Front-end. Através da implementação dos vários endpoints, foi possível visualizar as operações CRUD (Create, Read, Update, Delete), assegurando que a gestão de dados, tanto de utilizadores como de eventos, fosse realizada de forma persistente e segura.

- Melhorias possíveis:

Embora a aplicação se apresente estável e operacional, existem diversos pontos que podem ser explorados como evolução do projeto como implementar o resto dos endpoints desenvolvidos no backend e pesquisa e filtragem avançadas